

Abstraction + Inheritance + super()

1. Vehicle System (Easy)

Question

Create an abstract class Vehicle.

Parent Class:

- brand

Abstract Method:

- start()

Child Class:

- Car

Requirements:

- Use super() to initialize brand.
- Implement start().

Concept

Learn how super() calls the parent constructor and how abstraction forces the child class to implement methods.

2. Employee Management (Easy)

Question

Create an abstract class Employee.

Attributes:

- name
- salary

Abstract Method:

- work()

Child Class:

- Developer

Requirements:

- Use `super()` for name and salary.
- Implement `work()`.

Concept

Pass common data to the parent class using `super()`.

3. Bank Account (Easy)

Question

Create an abstract class `Bank`.

Attributes:

- `account_holder`
- `account_number`

Abstract Method:

- `show_interest_rate()`

Child Class:

- `SBI`

Requirements:

- Use `super()`.
- Print SBI interest rate.

Concept

Combine abstraction and constructor inheritance.

4. Animal Sound System (Easy)

Question

Create an abstract class `Animal`.

Attribute:

- `animal_name`

Abstract Method:

- `sound()`

Child Class:

- Dog

Requirements:

- Use super() to initialize animal name.
- Implement sound method.

Concept

Child class must implement abstract methods.

5. Shape Area Calculator (Easy)

Question

Create an abstract class Shape.

Attribute:

- color

Abstract Method:

- area()

Child Class:

- Rectangle

Requirements:

- Use super().
- Calculate rectangle area.

Concept

Use parent constructor and child-specific logic.

Medium Level

6. Online Payment System

Question

Create an abstract class Payment.

Attributes:

- user_name
- amount

Abstract Method:

- make_payment()

Child Classes:

- UPI
- CreditCard

Requirements:

- Use super().
- Implement payment logic.

Concept

Multiple child classes implementing the same abstract method.

7. School Management System

Question

Create an abstract class School.

Attributes:

- school_name
- city

Abstract Method:

- show_department()

Child Class:

- EngineeringCollege

Requirements:

- Use super().
- Display department details.

Concept

Constructor chaining with abstraction.

8. Food Delivery App

Question

Create an abstract class FoodDelivery.

Attributes:

- customer_name
- order_id

Abstract Methods:

- place_order()
- track_order()

Child Class:

- Swiggy

Requirements:

- Use super().
- Implement both methods.

Concept

Multiple abstract methods in one parent class.

9. AI Model Prediction System

Question

Create an abstract class Model.

Attributes:

- model_name
- version

Abstract Method:

- predict()

Child Class:

- RecommendationModel

Requirements:

- Use super().
- Implement prediction method.

Concept

Real-world AI abstraction example.

10. Hospital Management System

Question

Create an abstract class Hospital.

Attributes:

- hospital_name
- location

Abstract Method:

- show_specialization()

Child Class:

- CardiologyDepartment

Requirements:

- Use super().
- Display specialization details.

Concept

Practice abstraction with real-world hierarchy.

Interview Question 1

What is super()?

Answer:

super() is used to access the parent class methods and constructor from a child class.

Example:

```
class Parent:
    def __init__(self, name):
        self.name = name

class Child(Parent):
    def __init__(self, name):
        super().__init__(name)
```

Interview Question 2

Why do we use super()?

- Reuse parent code
 - Avoid duplicate code
 - Access parent constructor
 - Improve maintainability
-

Interview Question 3

Can we use Abstraction without super()?

Answer:

Yes.

But if the parent class has a constructor and you want to initialize parent attributes, super() is the recommended approach.

Practice Challenge

Create:

Abstract Class

Person

Attributes:

- name
- age

Abstract Method:

- show_role()

Child Classes

- Student
- Teacher

Use super() and implement show_role() differently in both classes.